

Lecture 09 - Particle-based Simulations

Part 1: SPH and Vortex methods

Georgii Oblapenko

1 Objectives

In this lecture, we will learn about particle-based (Lagrangian) discretizations of (fluid dynamic) equations:

1. Why?
2. What?
3. How?

2 Particle-based discretizations

2.1 Why?

In general, why would we want to discretize something with particles?

2.2 Pros and cons of particle methods

Pros:

- Easier to parallelize
- Easier to capture multi-phase phenomena
- Often more stable
- Sometimes the only method possible (DSMC)

Cons:

- Harder to parallelize (load balancing, tracking particles)
- Sometimes less stable (i.e. lost particles)
- More memory required

2.3 What

What kind of systems can we discretize with particles?

2.4 A different kind of particle discretization

Iannis Xenakis (1992-2001): mathematician, composer, architect. “Formalized Music: Thought and Mathematics in Composition” (1971).

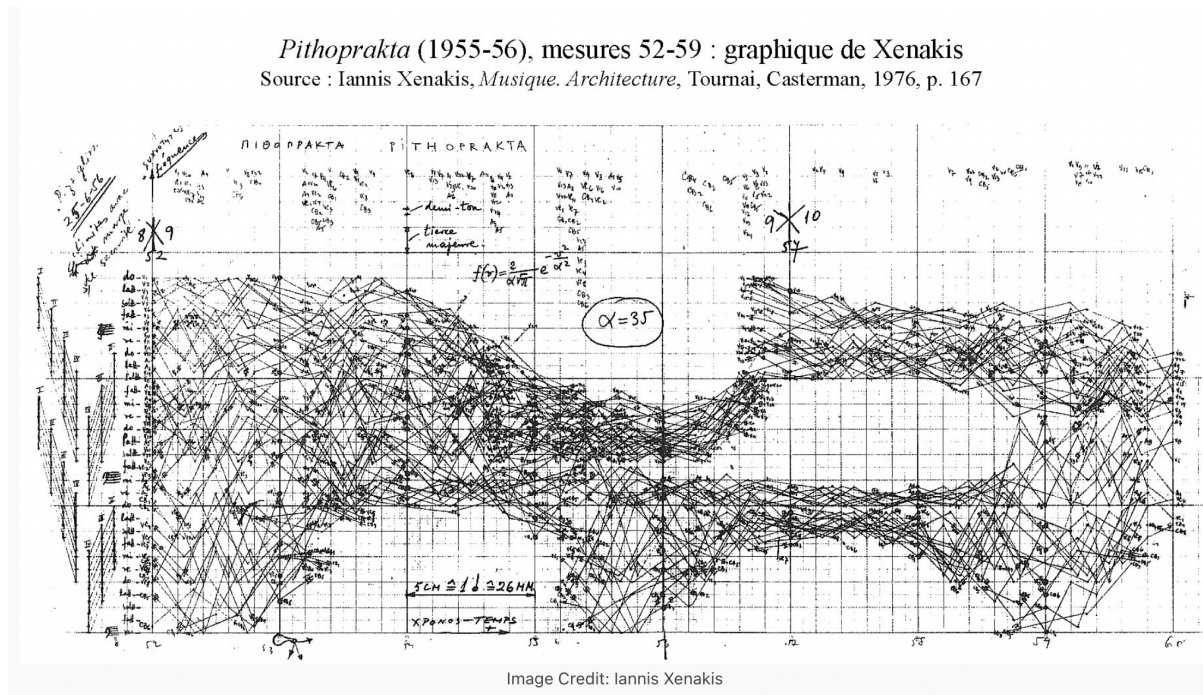


Figure 1: Iannis Xenakis: Pithoprakta graphical score

2.5 What kind of systems we will be discretizing

1. Weakly compressible fluids - SPH
2. Vortex methods
3. Rarefied gas flows - DSMC (next week)
4. Plasma flows - PIC (next week)

Other possible applications:

1. Radiative transport
2. Astrophysical flows (version of SPH)
3. Molecular dynamics

2.6 What is a particle?

$$p = (w, \mathbf{x}, \mathbf{v}, \mathbf{s}) \in \mathbb{R} \times \mathbb{R}^{d_x} \times \mathbb{R}^{d_v} \times \mathbb{P},$$

where $\mathbf{s} \in \mathbb{P}$ holds any additional information (for example, species index, temperature, etc.).

2.7 Components of a particle simulation

Physical:

- Particle creation
- Particle-particle interactions
- Particle movement
- Particle-surface interaction

Numerical: all of the above, and

- particle indexing
- particle sorting
- load balancing

2.8 Particle movement update

$$\mathcal{M} : (\mathbf{x}^n, \mathbf{v}^n) \rightarrow (\mathbf{x}^{n+1}, \mathbf{v}^{n+1}),$$

$\mathcal{M}$ - movement operator.

Forward Euler, no external forces:

$$(\mathbf{x}^n, \mathbf{v}^n) \rightarrow (\mathbf{x}^n + \Delta t \mathbf{v}^n, \mathbf{v}^n)$$

Forward Euler with $d_x < d_v$, no external forces:

$$(\mathbf{x}^n, \mathbf{v}^n) \rightarrow (\mathbf{x}^n + \Delta t (v_1^n, \dots, v_{d_v}^n), \mathbf{v}^n)$$

2.9 Particle movement - axisymmetric

Forward Euler with $d_x = 2, d_v = 3$, axisymmetric ($\mathbf{x} = (x, z)$):

1. $x' = x^n + v_x^n \Delta t, y = v_y^n \Delta t, z^{n+1} = z^n + v_z^n \Delta t$
2. $x^{n+1} = \sqrt{(x')^2 + y^2}$
3. $v_x^{n+1} = \frac{v_x x' + v_y y}{x^{n+1}}, v_y^{n+1} = \frac{v_y x' + v_x y}{x^{n+1}}, v_z^{n+1} = v_z^n$

3 SPH

Smoothed particle hydrodynamics (SPH)

Used for

- astrophysics: compressible flows with extreme gradients, no boundaries
- fluid flows: multi-phase weakly compressible flows
- computer graphics: real-time realistic rendering of fluids

3.1 Smoothing kernels

Particles are discrete objects, how do we represent a continuous field (for example, velocity)?

For a continuous field $f(\mathbf{x})$, we can write:

$$f(\mathbf{x}) = \int f(\mathbf{x}')\delta(\mathbf{x} - \mathbf{x}')d\mathbf{x}' \approx \int f(\mathbf{x}')W_h(\mathbf{x} - \mathbf{x}')d\mathbf{x}',$$

where $W_h(\mathbf{x})$ is an approximation of the delta function and is called a **smoothing kernel**. h is a support radius (although some kernels have infinite support radius).

3.2 Smoothing kernels: properties

From approximation of delta function property:

1. $\int W_h(\mathbf{x})d\mathbf{x} = 1$
2. $\lim_{h \rightarrow 0} W_h(\mathbf{x}) = \delta(\mathbf{x})$ (in the sense of measures)

We also impose:

3. $W_h(\mathbf{x}) = W_h(|\mathbf{x}|)$ (isotropy)
4. $W_h(\mathbf{x}) = 0$ for $|\mathbf{x}| > \kappa h$ (compact support) (*usually*)
5. $W_h(\mathbf{x}) > 0$ (*usually*)
6. $dW_h(|\mathbf{x}|)/d|\mathbf{x}| < 0$ (*usually*)
7. $W_h \in C^2$

3.3 Approximation error

What is the error in the approximation

$$f(\mathbf{x}) \approx \int f(\mathbf{x}')W_h(\mathbf{x}' - \mathbf{x})d\mathbf{x}' =: \langle f(\mathbf{x}) \rangle_h ?$$

If we assume

- sufficiently smooth f
- symmetric kernel W_h with compact support

then the error is of order h^2 . This is done by a Taylor expansion of f around $\mathbf{x}$.

3.4 Gradients

We are interested in solving PDEs, so we need to be able to compute gradients. We want to define the gradient of f by using the gradient of W .

In 1D:

$$\frac{df}{dx}(x) \approx \int_{-\infty}^{\infty} \frac{df}{dx}(x')W_h(x-x')dx' = W_h(x-x')f(x')|_{x'=-\infty}^{x'=\infty} - \int_{-\infty}^{\infty} f(x')\frac{dW_h}{dx'}(x-x')dx' = \int_{-\infty}^{\infty} f(x')\frac{dW_h}{dx}(x-x')d$$

3.5 Smoothing kernels: examples

Let's look at some common kernels. Defining $q = \frac{|x|}{h}$, we have:

- Gaussian

$$W(x, h) = \frac{1}{h\sqrt{\pi}} e^{-q^2}$$

- Cubic spline

$$W(x, h) = \frac{2}{3h} \begin{cases} 1 - \frac{3}{2}q^2 + \frac{3}{4}q^3 & 0 \leq q < 1 \\ \frac{1}{4}(2 - q)^3 & 1 \leq q < 2 \\ 0 & q \geq 2 \end{cases}$$

- Wendland (two-dimensional)

$$W(x, h) = \begin{cases} \frac{5}{8h} \left(1 - \frac{q}{2}\right)^3 \left(1 + \frac{3}{2}q\right) & 0 \leq q \leq 2 \\ 0 & q > 2 \end{cases}$$

3.6 Smoothing kernel: plots

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # 1. Kernel Definitions (1D)
6 # =====
7
8 def gaussian(x, h):
9     q = np.abs(x) / h
10    alpha = 1.0 / (h * np.sqrt(np.pi))
11    return alpha * np.exp(-q**2)
12
13 def grad_gaussian(x, h):
14     q = np.abs(x) / h
15     sign = np.sign(x)
16     alpha = 1.0 / (h * np.sqrt(np.pi))
17     # Derivative w.r.t x
18     return sign * (alpha / h) * (-2.0 * q) * np.exp(-q**2)
19
20 def cubic_spline(x, h):
21     q = np.abs(x) / h
22     alpha = 2.0 / (3.0 * h)
23     w = np.zeros_like(x)
24
25     mask1 = q < 1
26     w[mask1] = alpha * (1.0 - 1.5 * q[mask1]**2 + 0.75 * q[mask1]**3)
27
28     mask2 = (q >= 1) & (q < 2)
```

```

29     w[mask2] = alpha * 0.25 * (2.0 - q[mask2])**3
30
31     return w
32
33 def grad_cubic_spline(x, h):
34     q = np.abs(x) / h
35     sign = np.sign(x)
36     alpha = 2.0 / (3.0 * h)
37     dw = np.zeros_like(x)
38
39     mask1 = q < 1
40     dw[mask1] = sign[mask1] * (alpha / h) * (-3.0 * q[mask1] + 2.25 * q[mask1]**2)
41
42     mask2 = (q >= 1) & (q < 2)
43     dw[mask2] = sign[mask2] * (alpha / h) * (-0.75 * (2.0 - q[mask2])**2)
44
45     return dw
46
47 def wendland_c2(x, h):
48     q = np.abs(x) / h
49     alpha = 5.0 / (8.0 * h)
50     w = np.zeros_like(x)
51
52     mask = q < 2
53     w[mask] = alpha * (1.0 - q[mask]/2.0)**3 * (1.0 + 1.5 * q[mask])
54
55     return w
56
57 def grad_wendland_c2(x, h):
58     q = np.abs(x) / h
59     sign = np.sign(x)
60     alpha = 5.0 / (8.0 * h)
61     dw = np.zeros_like(x)
62
63     mask = q < 2
64     dw[mask] = sign[mask] * (alpha / h) * (-3.0 * q[mask] * (1.0 - q[mask]/2.0)**2)
65
66     return dw
67
68 # =====
69 # 2. Setup Plot Arrays
70 # =====
71 h = 1.0 # Smoothing length
72 x = np.linspace(-3, 3, 500)
73
74 # Calculate Kernels
75 w_gauss = gaussian(x, h)
76 w_cubic = cubic_spline(x, h)
77 w_wend = wendland_c2(x, h)
78
79 # Calculate Derivatives

```

```

80 dw_gauss = grad_gaussian(x, h)
81 dw_cubic = grad_cubic_spline(x, h)
82 dw_wend = grad_wendland_c2(x, h)
83
84 # =====
85 # 3. Plotting
86 # =====
87 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
88 fig.suptitle(f"Common SPH Kernels and their Derivatives (h={h})", fontsize=14)
89
90 # --- Plot Kernels ---
91 ax1.plot(x, w_gauss, label="Gaussian", color='blue', linewidth=2)
92 ax1.plot(x, w_cubic, label="Cubic Spline", color='red', linestyle='--', linewidth=2)
93 ax1.plot(x, w_wend, label="Wendland", color='green', linestyle=':', linewidth=2)
94
95 ax1.set_title("Kernel Function:  $W(x, h)$ ")
96 ax1.set_xlabel("Distance  $x$ ")
97 ax1.set_ylabel("Weight")
98 ax1.grid(True, alpha=0.3)
99 ax1.legend()
100 ax1.set_xlim(-2.5, 2.5)
101
102 # --- Plot Derivatives ---
103 ax2.plot(x, dw_gauss, label="Gaussian", color='blue', linewidth=2)
104 ax2.plot(x, dw_cubic, label="Cubic Spline", color='red', linestyle='--', linewidth=2)
105 ax2.plot(x, dw_wend, label="Wendland", color='green', linestyle=':', linewidth=2)
106
107 ax2.set_title("Kernel Spatial Gradient:  $\nabla W(x, h)$ ")
108 ax2.set_xlabel("Distance  $x$ ")
109 ax2.set_ylabel("Derivative")
110 ax2.grid(True, alpha=0.3)
111 ax2.legend()
112 ax2.set_xlim(-2.5, 2.5)
113
114 # Highlight the anti-symmetry
115 ax2.axhline(0, color='black', linewidth=1)
116 ax2.axvline(0, color='black', linewidth=1, linestyle='--')
117
118 plt.tight_layout()
119 plt.show()

```

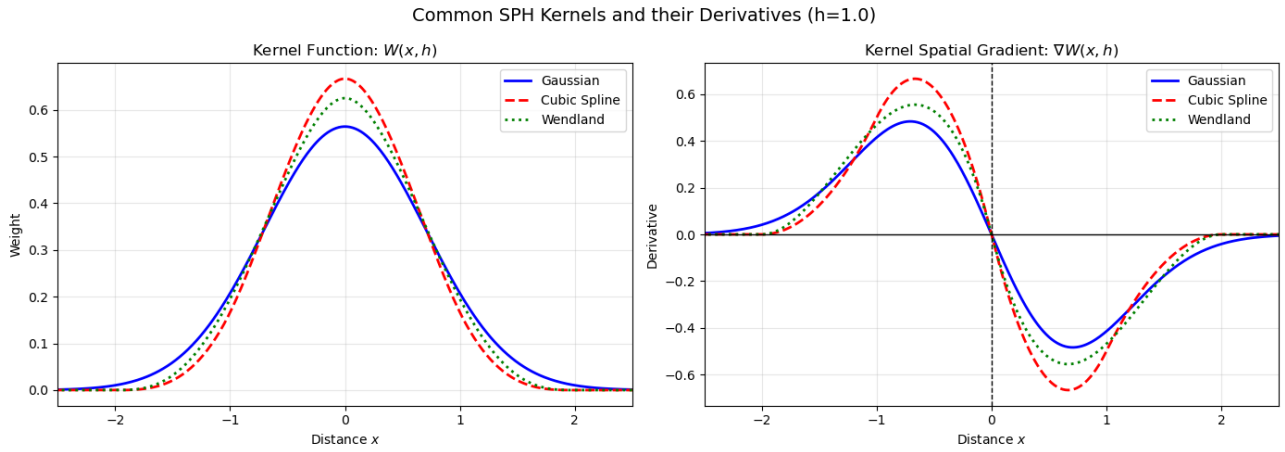


Figure 2: Kernel examples

3.7 Particle-based discretization

What is missing? We need to replace integration with summation over particles:

$$\mathbf{A}(\mathbf{x}) \approx \langle \mathbf{A}(\mathbf{x}) \rangle_h = \sum_j m_j \frac{A_j}{\rho_j} W_h(\mathbf{x} - \mathbf{x}_j), \quad \rho_j = \frac{m_j}{V_j}.$$

If we take our PDE, and set $\mathbf{x} = \mathbf{x}_i$, we will get PDEs for the particles' properties.

We will now

- omit the h subscript (and assume h is constant across particles)
- denote $W(|\mathbf{x}_i - \mathbf{x}_j|)$ as W_{ij}

3.8 Euler equations

We consider

$$\frac{d\rho}{dt} = -\rho \nabla \cdot \mathbf{v}, \quad \frac{d\mathbf{v}}{dt} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}$$

with a weakly compressible E.o.S.:

$$p = \frac{c_0^2 \rho_0^2}{\gamma} \left(\left(\frac{\rho}{\rho_0} \right)^\gamma - 1 \right),$$

c_0 is a numerical speed of sound: the larger, the better the approximation of incompressibility, but the smaller the possible timestep.

Estimate (leads to $c_0 = 10v_{max,flow}$ if $\delta\rho/\rho \approx 1\%$.)

$$c_0 = \frac{v_{max,flow}}{\sqrt{\delta\rho/\rho}}$$

3.9 Density equation discretization

We can simply write the definition of density:

$$\rho_i = \sum_j m_j W_{ij}.$$

No ODE to solve here, direct conservation of mass, no impact of velocity divergence on density.

But fails if there are not enough particles in the vicinity (free-surface flow or near boundaries).

3.10 Density equation discretization: ODE

We can write the density equation for a particle i as

$$\frac{d\rho_i}{dt} = -\rho_i \sum_j \frac{m_j}{\rho_j} \mathbf{v}_j \cdot \nabla_i W_{ij}.$$

Question

Why is this discretization not good?

In case 1) velocity is uniform and 2) particle distribution is asymmetric, the summation on the right-hand side will not be equal to 0. This will lead to a drift in density. Error due to particle discretization, not due to kernel (if kernel is symmetric).

3.11 Density equation discretization: symmetrization

We can write (on a continuous level)

$$\nabla A \approx \langle \nabla A \rangle = \langle \nabla A \rangle - A \langle \nabla 1 \rangle = (*)$$

and discretize the second equation:

$$(*) \approx \sum_j \frac{m_j}{\rho_j} A_j \cdot \nabla_i W_{ij} - A_i \sum_j \frac{m_j}{\rho_j} \cdot \nabla_i W_{ij}$$

3.12 Symmetrized density equation

We obtain for the density equation

$$\frac{d\rho_i}{dt} = -\rho_i \sum_j \frac{m_j}{\rho_j} (\mathbf{v}_j - \mathbf{v}_i) \cdot \nabla_i W_{ij}.$$

3.13 Momentum equation discretization

For momentum (without external forces and viscosity), we have

$$\frac{d\mathbf{v}}{dt} = -\frac{1}{\rho}\nabla p.$$

Directly applying SPH discretization, we get

$$\frac{d\mathbf{v}_i}{dt} = -\frac{1}{\rho_i} \sum_j \frac{m_j}{\rho_j} p_j \nabla_i W_{ij}.$$

Question

Why is this a bad discretization?

3.14 Newton's laws of motion

We have from Newton's second law:

$$\frac{d\mathbf{v}_i}{dt} = \mathbf{a}_i = \frac{1}{m_i} \mathbf{f}_i.$$

We can think of the summands as forces acting between particles i and j :

$$\mathbf{f}_{ij} = -\frac{m_i m_j}{\rho_i \rho_j} p_j \nabla_i W_{ij}.$$

Computing $\mathbf{f}_{ji}$, we have ($\nabla_i W_{ij} = -\nabla_j W_{ji}$)

$$\mathbf{f}_{ji} = -\frac{m_i m_j}{\rho_i \rho_j} p_i \nabla_j W_{ji} = \frac{m_i m_j}{\rho_i \rho_j} p_i \nabla_i W_{ij}.$$

Unless $p_i = p_j$, we don't have $\mathbf{f}_{ij} = -\mathbf{f}_{ji}$!

3.15 Symmetrized discretization

We can fix the issue by noticing that

$$\frac{1}{\rho} \nabla p = \nabla \left(\frac{p}{\rho} \right) + \frac{p}{\rho^2} \nabla \rho.$$

Discretizing both gradients via SPH, we get

$$\frac{d\mathbf{v}_i}{dt} = -\sum_j m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla_i W_{ij}.$$

The symmetrized factor $\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2}$ is what ensures consistency with Newton's third law.

3.16 1D two-particle example

We can see impact of the pressure gradient symmetrization on a simple 1D example with two particles.

```
1 from matplotlib.animation import FuncAnimation
2 from IPython.display import HTML
3
4 # =====
5 # 1. Setup Parameters
6 # =====
7 m = np.array([1.0, 1.0])      # Equal masses
8 rho = np.array([1.0, 1.0])    # Equal densities
9 P = np.array([1.0, 5.0])      # DIFFERENT pressures (Causes the asymmetry!)
10 h = 1.5                      # Smoothing length
11 dt = 0.05                    # Time step
12 steps = 80                   # Number of frames
13
14 # Derivative of a 1D Gaussian Kernel
15 def grad_W(xi, xj, h):
16     r = xi - xj
17     # dW/dx
18     return -2.0 * r / (h**2) * (1.0 / (np.sqrt(np.pi) * h)) * np.exp(-(r**2) / (h**2))
19
20 # Initialize trajectories
21 x_naive = np.zeros((steps, 2))
22 x_sym = np.zeros((steps, 2))
23
24 # Start particles at x = 5.0-0.5 and x = 5.0+0.5
25 x_naive[0] = np.array([5.0-0.5, 5.0+0.5])
26 x_sym[0] = np.array([5.0-0.5, 5.0+0.5])
27
28 v_naive = np.array([0.0, 0.0])
29 v_sym = np.array([0.0, 0.0])
30
31 # =====
32 # 2. Run the Simulation
33 # =====
34 for t in range(1, steps):
35
36     # Kernel gradients (anti-symmetric: dW_12 = -dW_21)
37     dW_12_naive = grad_W(x_naive[t-1, 0], x_naive[t-1, 1], h)
38     dW_21_naive = grad_W(x_naive[t-1, 1], x_naive[t-1, 0], h)
39
40     dW_12_sym = grad_W(x_sym[t-1, 0], x_sym[t-1, 1], h)
41     dW_21_sym = grad_W(x_sym[t-1, 1], x_sym[t-1, 0], h)
42
43     # --- NAIVE DISCRETIZATION (Violates 3rd Law) ---
44     # f_ij = - (m_i * m_j * P_j) / (rho_i * rho_j) * grad_W
45     f_naive_1 = - (m[0]*m[1]*P[1]) / (rho[0]*rho[1]) * dW_12_naive
46     f_naive_2 = - (m[1]*m[0]*P[0]) / (rho[1]*rho[0]) * dW_21_naive
47
```

```

48     # --- SYMMETRIC DISCRETIZATION (Obeyes 3rd Law) ---
49     #  $f_{ij} = -m_i * m_j * (P_i/\rho_i^2 + P_j/\rho_j^2) * \text{grad}_W$ 
50     P_term = (P[0]/rho[0]**2 + P[1]/rho[1]**2)
51     f_sym_1 = - m[0]*m[1] * P_term * dW_12_sym
52     f_sym_2 = - m[1]*m[0] * P_term * dW_21_sym
53
54     # Update velocities and positions (Euler integration)
55     v_naive += np.array([f_naive_1, f_naive_2]) / m * dt
56     x_naive[t] = x_naive[t-1] + v_naive * dt
57
58     v_sym += np.array([f_sym_1, f_sym_2]) / m * dt
59     x_sym[t] = x_sym[t-1] + v_sym * dt
60
61     # =====
62     # 3. Create the Animation
63     # =====
64     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 4), sharex=True)
65     fig.suptitle("SPH Momentum: Naive vs Symmetric Discretization")
66
67     # Set up the plot limits and aesthetics
68     for ax in (ax1, ax2):
69         ax.set_xlim(0, 10)
70         ax.set_ylim(-0.5, 0.5)
71         ax.get_yaxis().set_visible(False)
72         ax.axhline(0, color='gray', linestyle='--', alpha=0.5)
73
74     ax1.set_title("Naive (Self-Acceleration / Drift)", color='red')
75     ax2.set_title("Symmetric (Momentum Conserved)", color='green')
76
77     # Initialize scatter plots for particles and Center of Mass (CoM)
78     pts_naive, = ax1.plot([], [], 'ro', markersize=10, label='Particles')
79     com_naive, = ax1.plot([], [], 'rx', markersize=10, label='Center of Mass')
80
81     pts_sym, = ax2.plot([], [], 'go', markersize=10, label='Particles')
82     com_sym, = ax2.plot([], [], 'gx', markersize=10, label='Center of Mass')
83
84     ax1.legend(loc="upper right")
85     ax2.legend(loc="upper right")
86
87     def animate(i):
88         # Update Naive
89         pts_naive.set_data(x_naive[i], [0, 0])
90         com_naive.set_data([np.mean(x_naive[i])], [0])
91
92         # Update Symmetric
93         pts_sym.set_data(x_sym[i], [0, 0])
94         com_sym.set_data([np.mean(x_sym[i])], [0])
95
96         return pts_naive, com_naive, pts_sym, com_sym
97
98     anim = FuncAnimation(fig, animate, frames=steps, interval=40, blit=True)

```

```

99
100 # Render in Jupyter
101 plt.close() # Prevent static plot from showing
102 HTML(anim.to_jshtml())

```

<IPython.core.display.HTML object>

1D two-particle example

3.17 Boundary conditions

- repulsive forces/particles: easy to implement, kernel “breaks down” ($\int W dx < 1$)
- fixed fluid particles: computational overhead, sharp angles are harder to handle
- ghost (mirrored) particles: fewer additional particles required, complex geometry is hard to handle
- kernel-corrected methods: non-trivial to ensure stability, but perhaps most “elegant” method (still requires repulsive force)

3.18 Particle handling

Question

Why do we need a special way to handle particle indexing?

To avoid $\mathcal{O}(N^2)$ calculations. Make use of compact support of kernel.

- Idea 1: use background mesh (sorting particles into a cell is cheap(-ish)), we know which cells are within cut-off radius of each cell
- Idea 2: “Verlet lists”: store list of neighbours within compact support + within a “skin” distance r_s , update list every time the fastest particle has traveled further than $r_s/2$

Can combine both ideas (use Idea 1 for updating Verlet list in Idea 2).

3.19 Convergence

Apart from $\mathcal{O}(h^2)$ error, convergence depends on uniformity/non-uniformity of particle distribution; one result is:

Convergence is achieved when all the following hold

- $m_i \rightarrow 0$ (mass of each particle)
- $h \rightarrow 0$
- $N \rightarrow \infty$ (number of particles)
- $\mathcal{N} \rightarrow \infty$ (number of neighbours of each particle)

3.20 SPH Codes

Some open-source SPH codes:

- [Splishsplash](#)
- [TrixiParticles.jl](#)
- [SPHinXsys](#)
- [pysph](#)

4 Vortex point/particle methods

A method for incompressible inviscid two-dimensional Euler equations (3-d possible but more complex).

4.1 Vorticity

Vorticity is the curl of the flow velocity:

$$\boldsymbol{\omega} = \nabla \times \mathbf{u}.$$

From vector calculus we have:

$$\nabla \cdot \boldsymbol{\omega} = 0.$$

4.2 Helmholtz theorems

For a barotropic ($\rho = \rho(p)$) inviscid fluid we have

1. Fluid particles on a vortex line (field line of $\boldsymbol{\omega}$) remain on that line. The vortex line itself moves with the fluid.
2. Circulation $\Gamma = \int \boldsymbol{\omega} \cdot \mathbf{n} dS$ is conserved, i.e. $\frac{d\Gamma}{dt} = 0$.

Thinking about particles, we have:

1. Particles transporting vorticity move with the fluid
2. Vorticity of particles stays constant

4.3 Helmholtz decomposition

Any vector field $\mathbf{u} \in C^2$ can be decomposed into a curl-free and divergence-free part:

$$\mathbf{u} = -\nabla\Phi + \nabla \times \mathbf{A}.$$

If we have $\nabla \cdot \mathbf{u} = 0$ (incompressible flow) $\mathbf{A}$ uniquely determines $\mathbf{u}$ and vice versa ($\mathbf{A}$ is determined up to a constant factor).

$\mathbf{A}$ is related to ω via the vector Poisson equation:

$$\nabla^2 \mathbf{A} = -\omega.$$

To recover $\mathbf{u}$ from ω , we use the Biot-Savart law:

$$\mathbf{u}(\mathbf{x}) = \frac{1}{4\pi} \int_{\Omega} \frac{\omega(\mathbf{x}') \times (\mathbf{x} - \mathbf{x}')}{|\mathbf{x} - \mathbf{x}'|^3} d\mathbf{x}'.$$

4.4 Euler equations in vortex form

For an inviscid incompressible flow, the Euler equations can be re-written as

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \omega \cdot \nabla \mathbf{u}.$$

LHS corresponds to transport of vorticity (it is the d/dt derivative of ω), while RHS is the vortex stretching term.

4.5 Euler equations in vortex form: 2D

Why is 2D especially well-suited for vortex methods?

Because vorticity becomes a scalar value!

$$\mathbf{u} = (u_x, u_y, 0) \Rightarrow \omega = \left(0, 0, \frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y}\right) =: (0, 0, \omega).$$

This also means the stretching term disappears, and we obtain pure transport:

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = 0.$$

! Important

The Biot-Savart law in 2-D is slightly different (accounting for single component of vorticity as well):

$$\mathbf{u}(\mathbf{x}) = \frac{1}{2\pi} \mathbf{k} \times \int_{\Omega} \frac{(\mathbf{x} - \mathbf{x}')}{|\mathbf{x} - \mathbf{x}'|^2} \omega(\mathbf{x}') d\mathbf{x}'.$$

4.6 Particle discretization

Particles transport circulation Γ_i (or vorticity ω_i), and we get a system of ODEs for particle movement

$$\frac{d\Gamma_i}{dt} = 0, \frac{d\mathbf{x}_i}{dt} = \mathbf{u}(\mathbf{x}_i), \mathbf{u}(\mathbf{x}_i) = \frac{1}{2\pi} \mathbf{k} \times \sum_j \frac{(\mathbf{x}_i - \mathbf{x}_j)}{|\mathbf{x}_i - \mathbf{x}_j|^2} \Gamma_j.$$

4.7 Boundary conditions

Either reflection (like in SPH), or vorticity production BC in viscous flows (requires solution of integral equation).

4.8 Fast multipole method (FMM)

Unlike SPH, we need to consider all pair-wise interactions to compute

$$\frac{1}{2\pi} \mathbf{k} \times \sum_j \frac{(\mathbf{x}_i - \mathbf{x}_j)}{|\mathbf{x}_i - \mathbf{x}_j|^2} \Gamma_j$$

for all particles. Similar issues arise in molecular dynamics, astrophysics (gravity), charged particle physics (Coulomb force).

Two components:

1. Approximation of potential/force by a series expansion, equivalent to “clumping” distant particles together. Drops cost from $\mathcal{O}(N_{distant})$ to $\mathcal{O}(1)$ for each particle. But this does not work for near-field interactions.
2. Tree-based domain decomposition: nearby interactions are evaluated directly, distant interactions are approximated by FMM with varying degree of fidelity. Adds $\mathcal{O}(\log(N_p))$ overhead to each particles $\Rightarrow$ total cost is $\mathcal{O}(N_p \log(N_p))$.

5 Questions

- What are smoothing kernels, what properties should they have, and how does one compute gradients?
- How does one discretize the weakly compressible Euler equations with SPH, and what corrections are needed for the density and momentum equations?
- How does one obtain a particle discretization of the incompressible Euler equations in vorticity form, and why is 2D a special (and “nice”) case?

6 References

SPH:

- J.J. Monaghan, Smoothed Particle Hydrodynamics
- L. Di G. Sigalotti et al., A new insight into the consistency of the SPH interpolation formula

VPM:

- P. Spallart, Vortex methods for separated flows
- S. G. Voutsinas, Vortex methods in aeronautics: how to make things work
- L. Chen, Introduction to fast multipole methods